CSCI 611
Montana Pawek

Assignment 3: Small Object Detection Using YOLO

Dataset and Preprocessing

The dataset used in this project was the Mapillary Traffic Sign Dataset, as suggested by the assignment, which contains street-level images with traffic sign annotations. The annotations to the images were originally in JSON format, and were converted into YOLO text file format for training. To improve detection performance and simplify the classification task, the conversion also grouped all the different traffic sign categories together into a single class named traffic_sign, and separated images with annotations from those without. This conversion was done by a script generated by ChatGPT, and tested by myself to make sure it was done properly. The dataset consisted of approximately 52,000 images, but only around 42,000 images had their corresponding annotations. After preprocessing, it was determined that around 6,000 images contained at least one traffic sign, while the remaining images only contained images with things such as cars and roads. The dataset was then split into training, validation, and test sets in a 0.7-0.15-0.15 ratio. It should be noted that this resulted in a highly imbalanced dataset, with a large number of background images with no traffic signs compared to images containing traffic signs.

Challenges in Small Object Detection

Detecting traffic signs with the YOLO model presented several challenges. First, the small object size of the target made detection difficult, as the sign took up much less space than cars and people which the model was much better at detecting. This also compounded with the fact that due to the small size of these signs, they often contained much more limited detail in the image, making them harder to distinguish than windows and storefronts. The initial model also probably suffered from the fact that it attempted to identify which type of sign it saw, which would greatly increase the complexity of the task – especially for such small objects with so few details. Finally, the fact that so many images didn't have any signs in the first place likely made training difficult, even though those images were also necessary so the model wouldn't overfit and always try to find a sign in an image. These challenges made small object detection significantly more difficult for the model than detecting larger objects such as vehicles.

Experimental Results

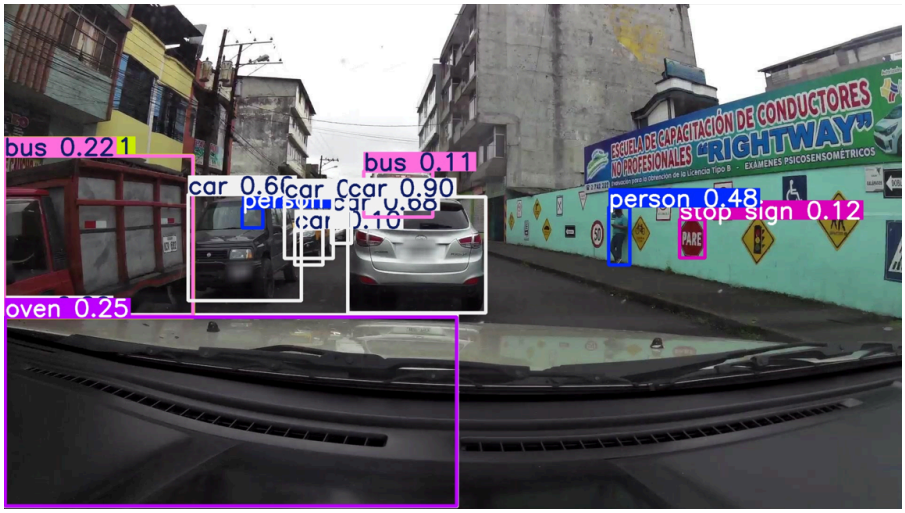

Figure 1: Baseline model failing to detect traffic signs

The pre-trained YOLOv8 model (yolov8n.pt) was used as a baseline for comparison; as can be seen in Figure 1, it was trained on general object detection datasets and was not specifically optimized for traffic sign detection as ours would hopefully be. After initial testing, the pre-trained model was able to successfully detect large objects such as cars and buses, but it frequently failed to detect small traffic signs, and produced low confidence scores for small/distant objects like signs. Figure 1 is a perfect example of this, as there are many traffic signs on the right side of the image that are not detected, but many cars and buses on the left are. This shows that standard object detection models struggle with small object detection without specific training.

Figure 2 and 3: Trained model only detects signs

The YOLO model was then fine-tuned using the prepared dataset to improve detection of traffic signs. Initially, the model was trained with 30 epochs, 960 image size, batch size 32, and the AdamW optimizer. After being measured against the validation set, the model yielded results of: Precision: 0.691, Recall: 0.450, mAP@50: 0.508, and mAP@50–95: 0.323. However, due to some unknown factor, all the training data, including weights, the annotated images

themself, and even the trained model vanished from my hard drive. After fruitlessly trying to find out what happened and restore them, I had to repeat the process again with a less rigorous model due to time constraints. The new training configuration was as follows: epochs: 10, image size: 640 (experimentation was run with 960 for testing as well), batch size: 4, and the AdamW optimizer. Data augmentation was applied during training using the default augmentation techniques provided by YOLO. These include random scaling, flipping, and color adjustments, which help improve generalization, especially for small object detection. Due to hardware and time constraints, and a loss of all initial preprocessed images and training data, the training parameters were adjusted to balance performance and training speed. Image resolution and batch size were reduced to avoid memory problems while still maintaining reasonable detection performance.

Figure 4: Example of annotated validation data

Figure 5: Example of model's sign identification

The validation results from the new training were as follows: Precision: 0.5003, Recall: 0.30040, mAP@50: 0.29274, and mAP@50-95: 0.16373. These results show that the fine-tuned model was able to learn the traffic sign detection task somewhat well - especially given the appropriate amount of hyperparameters - and was still capable of detecting small and distant objects with smaller epoch, image, and batch sizes as can be seen in Figures 2 and 3. Precision was noticeably higher than recall in both cases, indicating that the model's predictions were more often correct when they were made, but that many traffic signs were still missed or the model was not confident enough to identify the sign, as can be seen in Figures 4 and 5 – only one sign was detected across 7 images, 6 of which had signs.

The trained model was then evaluated on the test dataset using YOLO's evaluation function. The following performance results were obtained: Precision: 0.702, Recall: 0.252, mAP@50: 0.484, mAP@50–95: 0.315. These results indicate that the model produces relatively accurate detections when it identifies traffic signs, as reflected by the higher precision value. However, the lower recall suggests that many traffic signs, particularly small or distant ones, are still missed. This can be seen below in Figure 4, where achieving a recall value of around 0.5 appears to sacrifice almost all precision. Overall, the fine-tuned model demonstrates an improvement over the baseline in detecting traffic signs, but can still be improved. Different confidence thresholds of 0.25 and 0.10 were tested. Lowering the confidence threshold increased the number of detected traffic signs, improving recall, but this also introduced more false positives, as weaker predictions were included. This demonstrates the tradeoff between recall and precision when adjusting the confidence threshold. Perhaps a value between the two would be best, as the difference between the precision and recall stayed very high for most tests. The model was trained using an image size of 640, with testing of images of size 960. Higher image resolution improved the ability to detect small objects because more pixel information is preserved. However, increasing resolution significantly increases training time and memory usage.

The trained model was applied to a sequence of images from the test dataset to simulate video-like input. Across consecutive images, the model demonstrated improved consistency compared to the baseline model. Traffic signs were detected more reliably across multiple frames, although some small or distant signs were still missed. These results show that the trained model generalizes better across sequential data and performs more consistently in realistic driving scenarios.

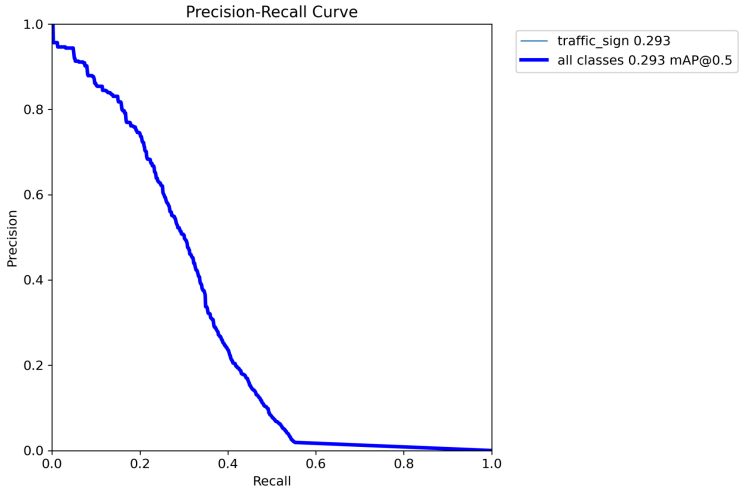

Figure 4: PR Curve

Improvement Suggestions

Several improvements could further enhance traffic sign detection performance. First, something as simple as using a larger YOLO model could improve detection accuracy, at the cost of increased computation. The image resolution can also improve detection of small objects when increased, but it also requires more memory and processing time. As usual, expanding the dataset or including additional annotated images would improve generalization, especially if the ratio of background images to images-with-signs was changed to favor the training. It would also be worth investigating if the annotations or bounding boxes could be better, which would help with identifying small and blurry signs. Finally, increasing the training parameters such as number of epochs, image size, and batch size would also likely yield better traffic sign detection.